Don Bosco Institute of Technology, Kurla
Academic Year 2025-26
EXPERIMENT NO: 1

Name of the student: Shraddha Gajanan Desai **Roll No: 18**

Division: A **Batch: A**

Aim of the Experiment: Design and Implementation of a product cipher using Substitution and Transposition.

Learning Objectives: To understand the encryption and decryption fundamentals.
To understand the concepts of the product cipher.

Lab Outcomes: Understand the principles and practices of cryptographic techniques.

Tools & Software Required: Python

Theory:
(Explain following terms with one example)

- **Substitution cipher:**

A Substitution Cipher is a method of encryption where each character in the plaintext is replaced by another character, number, or symbol according to a fixed rule. The "identity" of the letter changes, but its position in the sentence stays exactly the same.

- Example: Using a rule where every letter is replaced by its keyboard neighbor to the right.
  Plaintext: HE
  Ciphertext: JR (H becomes J, E becomes R)

- **Transposition cipher:**

A Transposition Cipher does not change the identity of the letters. Instead, it shuffles their positions. Think of it as a sophisticated anagram. The characters are exactly the same as the original, but their order is scrambled so the message cannot be read.

- Example: A simple "Reverse" transposition.

  Plaintext: HELLO

  Ciphertext: OLLEH

- **Caesar Cipher:**

The Caesar Cipher is the most famous type of substitution cipher. It works by shifting the alphabet by a certain number of positions. If the shift is 3, 'A' becomes 'D', 'B' becomes 'E', and so on.

- Example: Shift of 3.

  Plaintext: CAT

  Calculation: C+3=F, A+3=D, T+3=W

  Ciphertext: FDW

- **Columnar Cipher:**

A **Columnar Cipher** is a specific type of transposition cipher. The message is written horizontally into a grid (matrix) of a certain width and then read out vertically column-by-column to scramble the text.

- Example: Using 3

  columns. Plaintext:

  CIPHER Matrix:

  C I P

  H E R

  Reading Columns: Col 1 (CH), Col 2 (IE), Col 3 (PR)

  Ciphertext: CHIEPR

Algorithm/Procedure:

Part A: Caesar Cipher (Substitution)

1. Accept plaintext from the user.
2. Accept Caesar key from the user.
3. Take k = 0.
4. Extract the kth character from the plaintext string.
5. Add the key to the extracted character and obtain a new value.

   If the new value > 26,

New value = New value % 26.

6. Add the obtained character as the kth character of the intermediate ciphertext.
7. Increment k.
8. If (k < length(plaintext)) go to step 4.
9. Store the result as intermediate ciphertext.

Part B: Columnar Transposition Cipher

1. Accept columnar key (number of columns) from the user.
2. Write the intermediate ciphertext row-wise into a matrix using the columnar key.
3. Read the characters column-wise from the matrix.
4. Store the obtained characters as final ciphertext.
5. Display plaintext and final ciphertext (output).

Source Code:

```
import math

def product_cipher_full_trace():
    # Input
    raw_input_text = input("Enter the plaintext: ")
    plaintext = raw_input_text.upper().replace(" ", "")
    key = int(input("Enter the key (used for Caesar shift & columns): "))

    print("\nENCODING")

    # Step 1: Caesar Cipher
    print(f"\n[Step 1] Caesar Cipher (Shift: +{key})")
    print(f"{'Char':<6} | {'Calculation':<25} | {'Result'}")
    print("-" * 50)

    intermediate_ciphertext = ""
    for char in plaintext:
        original_ascii = ord(char)
        new_ascii = (original_ascii + key - 65) % 26 + 65
        result_char = chr(new_ascii)

        print(f"{char:<6} | ({original_ascii} + {key} - 65) % 26 + 65 |
{result_char}")
        intermediate_ciphertext += result_char

    # Step 2: Transposition
    num_cols = key
    num_rows = math.ceil(len(intermediate_ciphertext) / num_cols)

    padding_len = (num_rows * num_cols) - len(intermediate_ciphertext)
    padded_text = intermediate_ciphertext + ("X" * padding_len)
```

```
    matrix = [
        list(padded_text[i:i + num_cols])
        for i in range(0, len(padded_text), num_cols)
    ]

    print(f"\n[Step 2] Transposition Matrix ({num_rows}x{num_cols})")
    for row in matrix:
        print("  " + " | ".join(row))

    # Step 3: Read Columns
    final_ciphertext = ""
    print("\n[Step 3] Reading Columns for Final Ciphertext:")
    for c in range(num_cols):
        col_str = "".join(matrix[r][c] for r in range(num_rows))
        print(f"  Col {c + 1}: {col_str}")
        final_ciphertext += col_str

    print(f"\nENCRYPTED OUTPUT: {final_ciphertext}")

    print("\nDECODING")

    # Reverse Transposition
    print(f"\n[Step 1] Filling Matrix by Columns")
    dec_matrix = [['' for _ in range(num_cols)] for _ in range(num_rows)]

    idx = 0
    for c in range(num_cols):
        for r in range(num_rows):
            dec_matrix[r][c] = final_ciphertext[idx]
            idx += 1

    for row in dec_matrix:
        print("  " + " | ".join(row))

    caesar_restored = "".join("".join(row) for row in dec_matrix)
    print(f"\nTransposition Reversed (Row-wise): {caesar_restored}")

    # Reverse Caesar
    print(f"\n[Step 2] Reverse Caesar (Shift: -{key})")
    print(f"{'Char':<6} | {'Calculation':<25} | {'Result'}")
    print("-" * 50)

    decoded_plaintext = ""
    for char in caesar_restored:
        curr_ascii = ord(char)
        orig_ascii = (curr_ascii - key - 65) % 26 + 65
        result_char = chr(orig_ascii)

        print(f"{char:<6} | ({curr_ascii} - {key} - 65) % 26 + 65 | {result_char}")
        decoded_plaintext += result_char

    print(f"\nDECODED OUTPUT: {decoded_plaintext}")


if __name__ == "__main__":
    product_cipher_full_trace()
```

Output Screenshots:

Case 1:

```
C:\Users\Shraddha\PycharmProjects\PythonProject\.venv1\Scripts\python.exe D:\PythonProject\main.py
Enter the plaintext: meet me at noon
Enter the key (used for Caesar shift & columns): 3

ENCODING

[Step 1] Caesar Cipher (Shift: +3)
Char   | Calculation               | Result
-----------------------------------------------
M      | (77 + 3 - 65) % 26 + 65 | P
E      | (69 + 3 - 65) % 26 + 65 | H
E      | (69 + 3 - 65) % 26 + 65 | H
T      | (84 + 3 - 65) % 26 + 65 | W
M      | (77 + 3 - 65) % 26 + 65 | P
E      | (69 + 3 - 65) % 26 + 65 | H
A      | (65 + 3 - 65) % 26 + 65 | D
T      | (84 + 3 - 65) % 26 + 65 | W
N      | (78 + 3 - 65) % 26 + 65 | Q
O      | (79 + 3 - 65) % 26 + 65 | R
O      | (79 + 3 - 65) % 26 + 65 | R
N      | (78 + 3 - 65) % 26 + 65 | Q
```

```
[Step 2] Transposition Matrix (4x3)
  P | H | H
  W | P | H
  D | W | Q
  R | R | Q

[Step 3] Reading Columns for Final Ciphertext:
  Col 1: PWDR
  Col 2: HPWR
  Col 3: HHQQ

ENCRYPTED OUTPUT: PWDRHPWRHHQQ
```

Case 2:

```
DECODING

[Step 1] Filling Matrix by Columns
  P | H | H
  W | P | H
  D | W | Q
  R | R | Q

Transposition Reversed (Row-wise): PHHWPHDWQRRQ

[Step 2] Reverse Caesar (Shift: -3)
Char   | Calculation               | Result
-------------------------------------------------
P      | (80 - 3 - 65) % 26 + 65 | M
H      | (72 - 3 - 65) % 26 + 65 | E
H      | (72 - 3 - 65) % 26 + 65 | E
W      | (87 - 3 - 65) % 26 + 65 | T
P      | (80 - 3 - 65) % 26 + 65 | M
H      | (72 - 3 - 65) % 26 + 65 | E
D      | (68 - 3 - 65) % 26 + 65 | A
W      | (87 - 3 - 65) % 26 + 65 | T
Q      | (81 - 3 - 65) % 26 + 65 | N
R      | (82 - 3 - 65) % 26 + 65 | O
R      | (82 - 3 - 65) % 26 + 65 | O
Q      | (81 - 3 - 65) % 26 + 65 | N

DECODED OUTPUT: MEETMEATNOON
```

z

Conclusion :

This experiment proves that two ciphers are better than one. By using **Substitution** to change the letters and **Transposition** to scramble their positions, we created a "Product Cipher" that is much harder to break. The code successfully showed that if you follow the steps forward to encrypt, you can follow them perfectly backward to get the original message back.